



جامعة سيئون
كلية الحاسبات

ملزمة جامعية · البرمجة كائنية التوجه بلغة Java

التجريد في البرمجة
(Abstraction)

مفهوم Abstraction | الكلمة abstract | Abstract Class | Abstract Methods

Concrete Methods | الوراثة من Abstract Class | قيود إنشاء Objects | تطبيق التجريد

إعداد الطالب
حسين جمال الليل بن عقيل

لغة الأمثلة: Java
بيئة العمل: VS Code

1 مفهوم التجريد
What is Abstraction?

التجريد (Abstraction) هو أننا **نُظهر المهم فقط** للي يستخدم الشيء، و**نخفي التفاصيل الداخلية** المعقدة. يعني نركّز على **"ماذا يعمل؟"** بدل **"كيف يعمل من الداخل؟"**.

بالعامي

أنت لما تسوق السيارة، تستخدم الدركسون والفرامل والبنزين وبس. ما يهمك كيف المحرك يحرق الوقود ولا كيف الجير يتنقل. السيارة "جرّدت" لك الموضوع وعطتك اللي تحتاجه بس.

مثال السيارة كمخطط



السهم المتقطع = جزء مخفي عن المستخدم

وفي البرمجة؟

نفس الفكرة بالضبط: نكتب **كلاس عام** يحدد **"إيش لازم يكون موجود"** (مثلاً: كل شكل لازم له مساحة)، ونترك **"كيف تنحسب"** لكل كلاس فرعي حسب نوعه. الدائرة تحسبها بطريقة، والمستطيل بطريقة ثانية.

ليش التجريد مهم؟

- **يبسّط البرنامج:** اللي يستخدم الكلاس يشوف شي بسيط وواضح.
- **يوحّد الشكل:** كل الكلاسات الفرعية مُلزمة بنفس الدوال، فما أحد ينسى شي مهم.
- **يسهّل التعديل والتوسعة:** تضيف شكل جديد (مثل مثلث) بدون ما تلمس الكود القديم.
- **يقلل الأخطاء:** التفاصيل مخفية، فما أحد يعبث فيها بالغلط.

التعريف باختصار

التجريد = إخفاء التفاصيل + إظهار الأساسيات. وفي Java نطبّقه بشكل رئيسي عن طريق **Abstract Class** و **Interface**، وفي هذي الملزمة نركّز على Abstract Class.

2 الكلمة abstract و Abstract Class

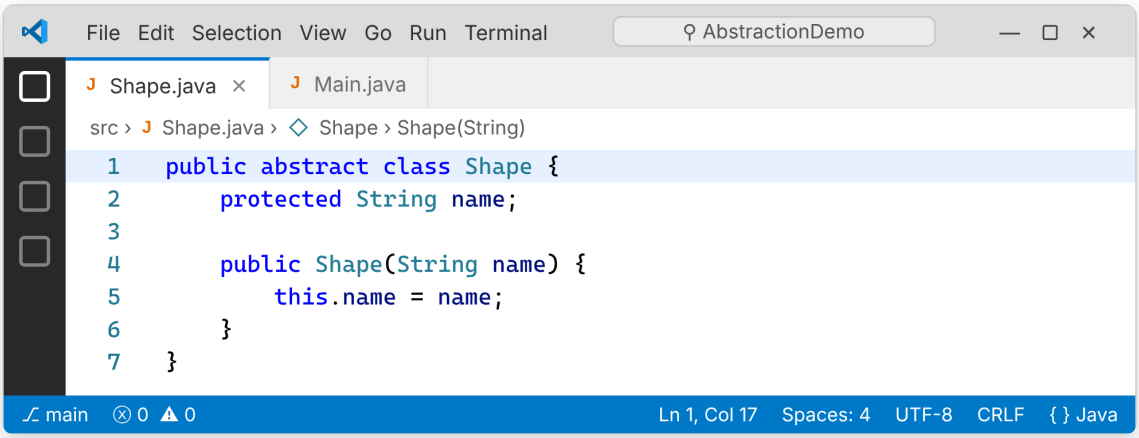
The abstract keyword

الكلمة المحجوزة `abstract` معناها **"ناقص / غير مكتمل"**. لما نحطها قبل `class` يصير عندنا **Abstract Class**: كلاس يمثّل **فكرة عامة**، ويُستخدم **كأساس** لكلاسات ثانية ترث منه.

بالعامي

كلمة "شكل" لحالها فكرة عامة. ما في شي اسمه "شكل" وبس! في دائرة، في مستطيل، في مثلث. فـ Shape هو "القالب العام"، والأشكال الحقيقية هي اللي تطلع منه.

طريقة الكتابة



كود (1): تعريف كلاس مجرد بسيط باسم Shape

شرح الكود

1 كلمة `abstract` قبل `class` هي اللي تخلي Shape كلاس مجرد.

2 عادي يكون فيه متغيرات (Fields) مثل `name` ، و `protected` تعني أن الأبناء يقدرون يوصلون له.

4–6 وعادي يكون فيه Constructor، يشتغل لما ننشئ كائن من كلاس **فرعي** يرث منه.

في Java كل كلاس `public` ينكتب في ملف بنفس اسمه، لذلك الملف اسمه `Shape.java` .

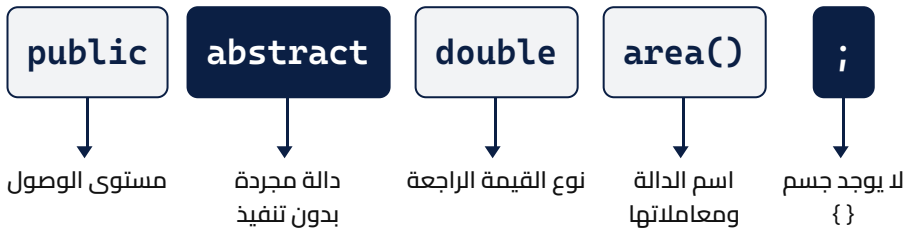
قاعدة مهمة

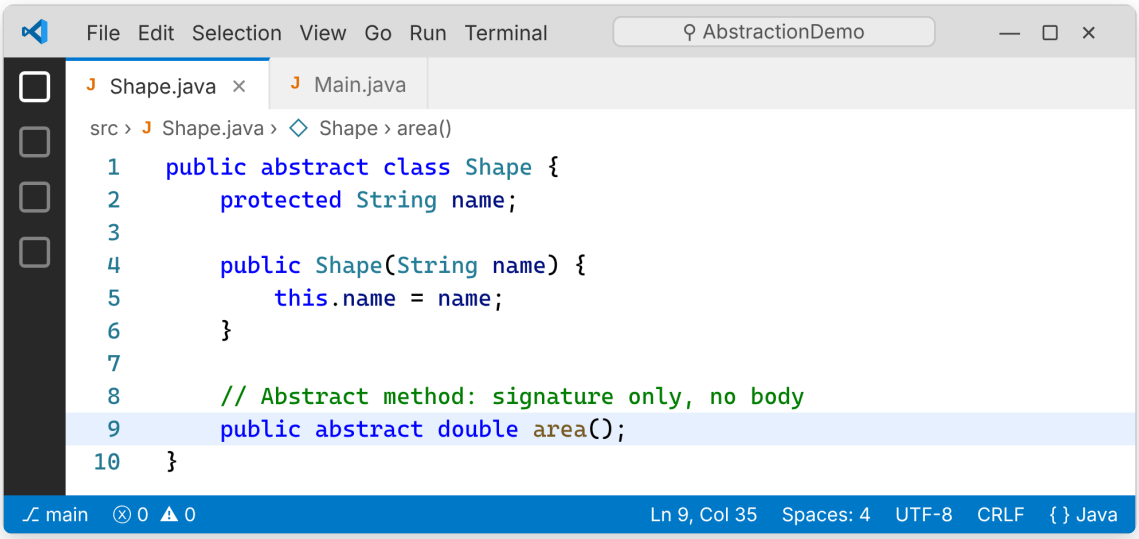
الـ Abstract Class كلاس طبيعي في كل شي تقريبًا (متغيرات، Constructor، دوال)، **الفرق الوحيد** أنه **ما ينفع ننشئ منه كائن مباشرة**، وممكن يحتوي على دوال مجردة (نشوفها في الصفحة الجاية).

3 الدوال المجردة
Abstract Methods

Abstract Method هي دالة **نكتب توقيعها فقط** (الاسم ونوع الإرجاع والمعاملات) **بدون جسم** `{ }`، وننهيها بفاصلة منقوطة `;`. هي عبارة عن **"وعد"**: كل كلاس يرث لازم يكتب جسمها بنفسه.

تشريح السطر





كود (2): إضافة دالة مجردة area() إلى Shape

شرح الكود

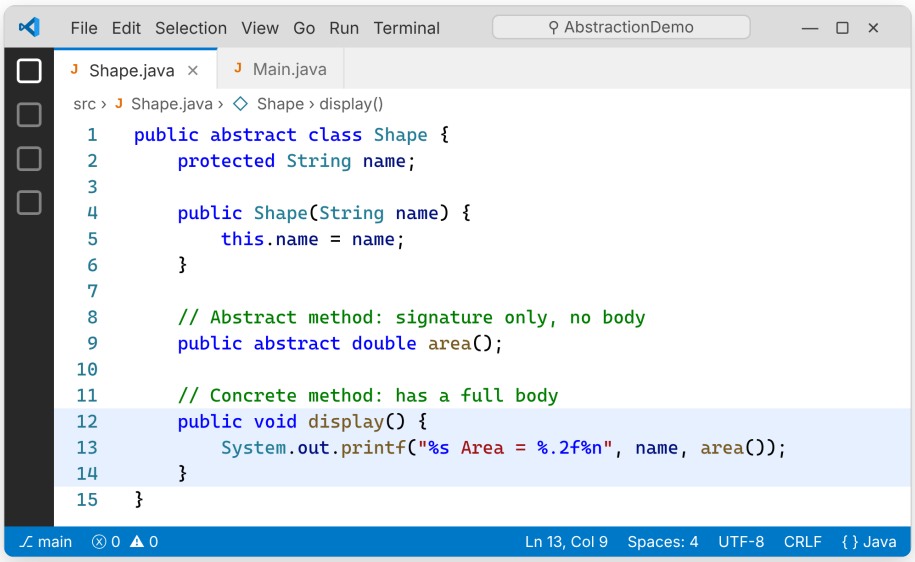
`9` `public abstract double area();` ← نقول: "كل شكل **لازم** يعرف يحسب مساحته"، بس ما نقول **كيف**؛ لأن طريقة الحساب تختلف من شكل لشكل.

شروط الدالة المجردة

- ما تنكتب إلا **داخل Abstract Class**. لو حطيتها في كلاس عادي يطلع خطأ.
- **ما لها جسم أبدًا**؛ لو كتبت لها `{ }` يعطيك المترجم خطأ.
- الكلاس الفرعي (غير المجرد) **مُلزَم** يعيد كتابتها بجسم كامل (Override)، ويُفضّل نكتب فوقها `@Override`.

4 الدوال المكتملة
Concrete Methods

Concrete Method هي دالة **عادية لها جسم كامل**. الـ Abstract Class يقدر يجمع النوعين: دوال مجردة (يكملها الأبناء) + دوال مكتملة (يستخدمها الأبناء جاهزة بدون ما يعيدوا كتابتها).



كود (3): دالة مكتملة display() داخل الكلاس المجرد

شرح الكود

9 `area()` مجردة: كل شكل يحسبها بطريقته.

12–14 `display()` مكتملة: تطبع اسم الشكل ومساحته. لاحظ إنها **تنادي** `area()` مع إنها مجردة! وقت التشغيل تتنفذ نسخة الكلاس الحقيقي (دائرة أو مستطيل).

13 `printf` للطباعة المنسّقة: `%s` مكان النص، و `%.2f` رقم برقمين بعد الفاصلة، و `%n` سطر جديد.

مقارنة سريعة

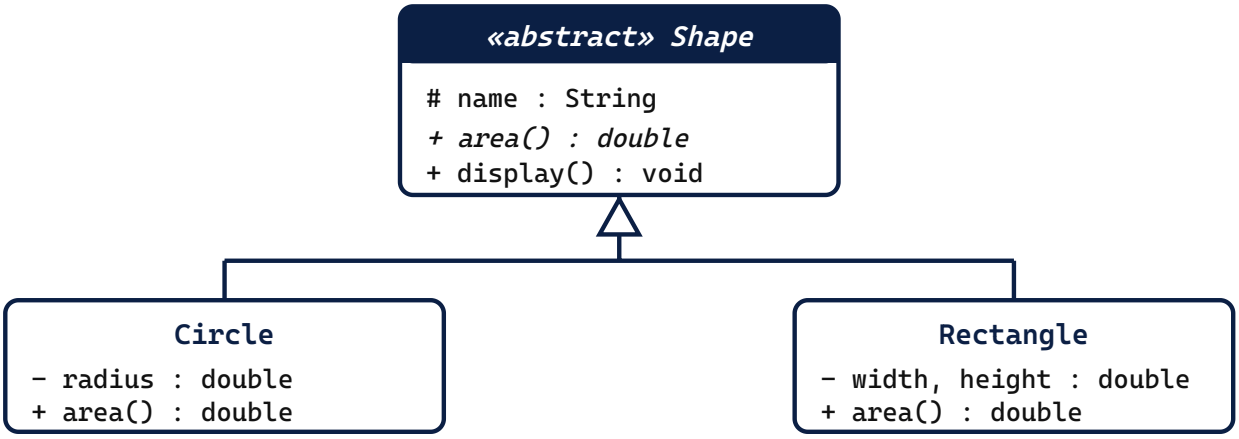
وجه المقارنة	Abstract Method	Concrete Method
الجسم { }	لا يوجد، تنتهي بـ `;`	موجود وكامل
الكلمة المميزة	`abstract`	لا شيء (دالة عادية)
مكانها	داخل Abstract Class فقط	أي كلاس
في الكلاس الفرعي	إلزامي تعيد كتابتها مع `@Override`	تُورث وتُستخدم جاهزة

5 الوراثة من Abstract Class

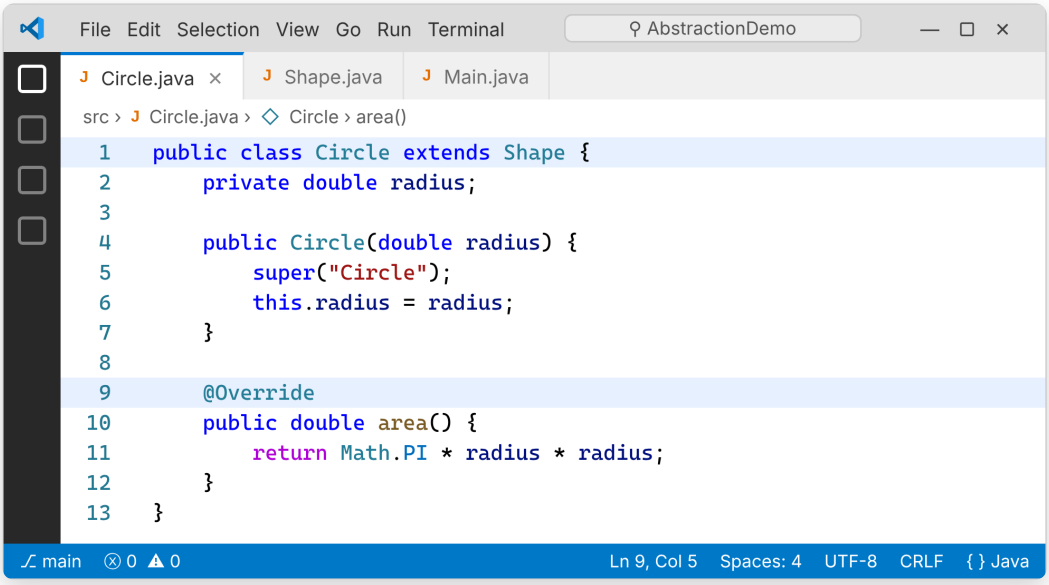
Inheriting from an Abstract Class

الـ Abstract Class ما له فايدة لحاله؛ فايدته لما **نورّث منه**. في Java نكتب الكلمة `extends` بعد اسم الكلاس الفرعي، ثم نعيد كتابة كل دالة مجردة بجسم كامل، ونكتب فوقها `@Override` .

مخطط الوراثة



السهم ذو الرأس المثلث يعني "يرث من" · الخط المائل في area() يعني دالة مجردة



كود (4): الكلاس Circle يرث من Shape

شرح الكود

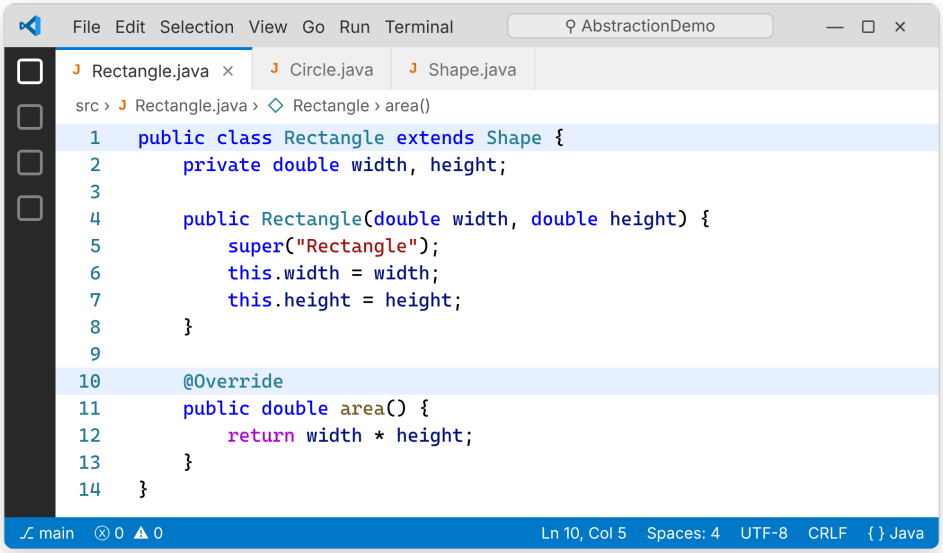
1 `Circle extends Shape` ← الدائرة ترث من Shape المتغير `name` والدالة `display()` .

5 `super("Circle")` ← تنادي Constructor الأب وتعطيه اسم الشكل، ولازم يكون أول سطر.

9–12 `@Override` ← هنا نكتب "الطريقة" اللي وعدنا فيها: مساحة الدائرة $\pi \times r^2$.

كلاس ثاني يرث من نفس الأب

بنفس الطريقة نكتب المستطيل. لاحظ: **نفس الدالة** `area()` لكن **بتنفيذ مختلف**. هذا هو جوهر التجريد.



كود (5): الكلاس Rectangle يرث من Shape

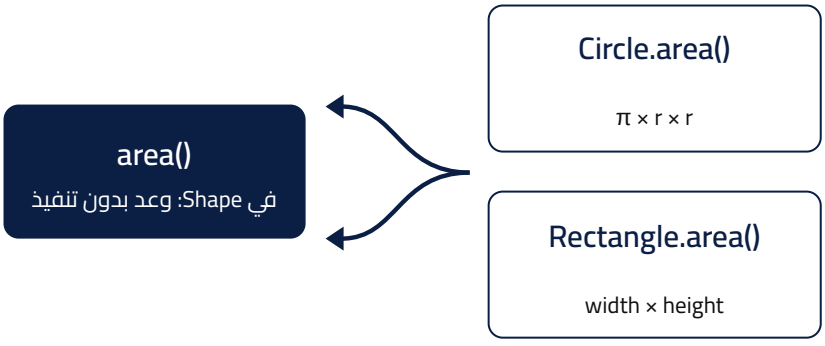
شرح الكود

1 `Rectangle extends Shape` ← المستطيل يرث من نفس الأب.

4–8 الـ Constructor يستقبل العرض والطول، و `super("Rectangle")` يرسل الاسم للأب.

10–13 مساحة المستطيل = العرض × الطول.

نفس الطلب.. وتنفيذ مختلف



انتبه: ماذا لو نسيت كتابة الدالة المجردة؟

لو كتبت `class Circle extends Shape` وما كتبت دالة `area()`، المترجم يرفض ويعطيك الخطأ:

```
Circle is not abstract and does not override abstract method area() in Shape
```

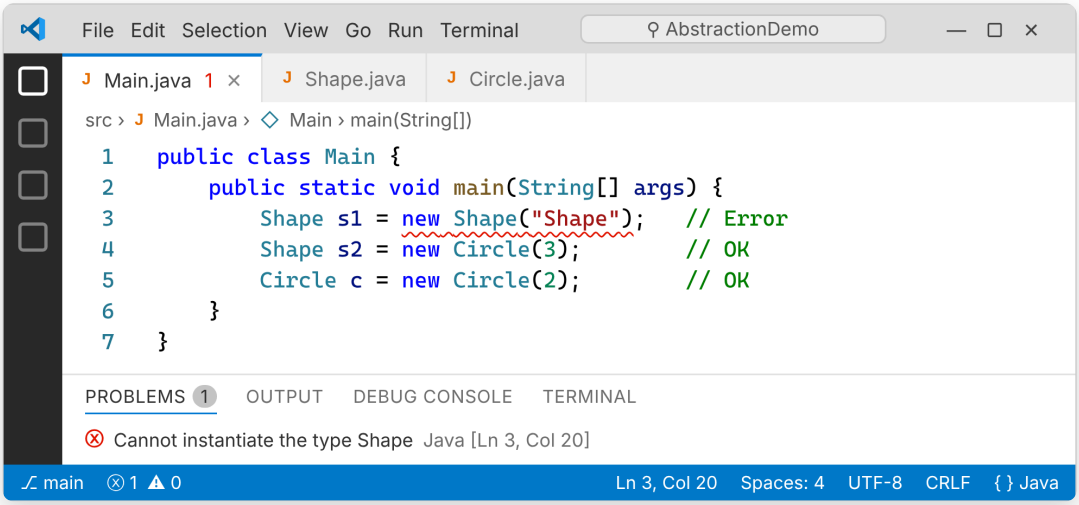
الحل: إما تكتب الدالة `area()` في Circle، أو تخلي Circle نفسه `abstract`.

6 قيود إنشاء Objects من Abstract Class
Instantiation Rules

ما نقدر ننشئ كائن من Abstract Class باستخدام `new`؛ لأنه ناقص (فيه دوال بدون تنفيذ). لكن **نقدر** نستخدمه **كنوع للمتغير** ونخزّن فيه كائن من كلاس فرعي.

بالعامي

لو قلت للنجار "سوّ لي **أثاث**"، بيسألك: "أي أثاث؟ كرسي ولا طاولة؟". ما يقدر يصنع "أثاث" بشكل عام! بس يقدر يصنع كرسي، والكرسي **هو** أثاث.



كود (6): المحاولة الخاطئة والمحاولات الصحيحة، مع رسالة الخطأ في PROBLEMS

شرح الكود

3 `new Shape(...)` ✗ خطأ: Cannot instantiate the type Shape، يعني ما يمكن إنشاء كائن من نوع مجرد (الخط الأحمر المتعرّج).

4 `Shape s2 = new Circle(3);` ✓ صحيح: المتغير نوعه Shape لكن الكائن الحقيقي دائرة.

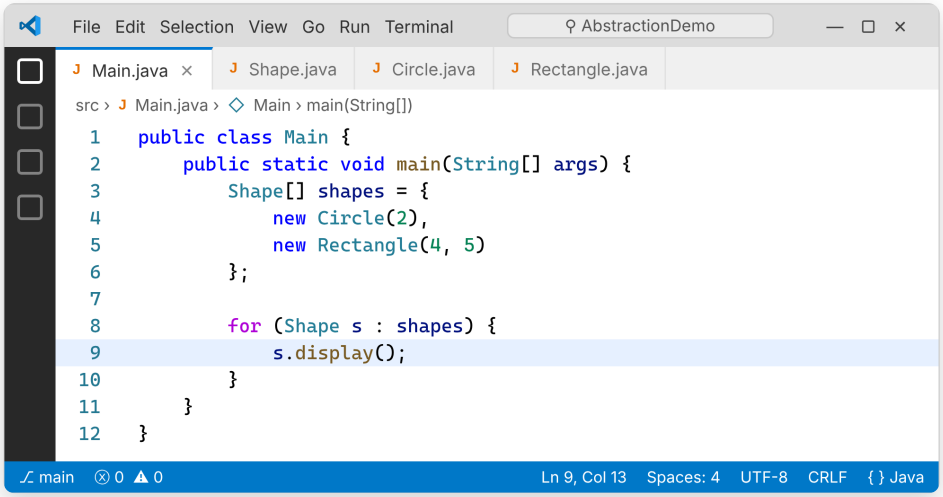
5 `Circle c = new Circle(2);` ✓ صحيح: كلاس عادي مكتمل.

كيف يبدو السطر 4 في الذاكرة؟



7 تطبيق التجريد في تصميم البرامج
Putting it all together

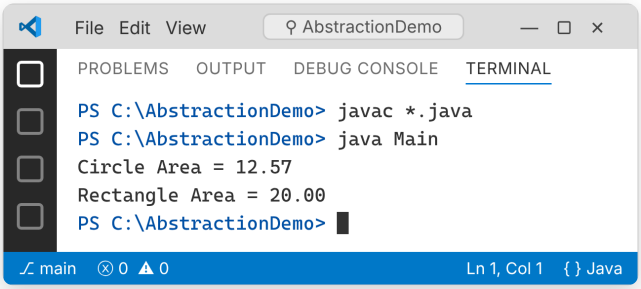
الآن نجمع كل شي: مصفوفة واحدة من نوع Shape فيها أشكال مختلفة، ونتعامل معها كلها **بنفس الطريقة** بدون ما نهتم بنوع كل شكل.



كود (7): البرنامج الرئيسي

تسلسل التنفيذ

1. يبدأ التنفيذ من main()
2. إنشاء مصفوفة Shape[] فيها دائرة ومستطيل
3. for-each يمر على كل عنصر
4. ينادي display() الموروثة من الأب
5. display() تنادي area() الخاصة بالكائن الحقيقي ثم تطبع



ناتج التشغيل في TERMINAL داخل VS Code

شرح الكود والناتج

3–6 المصفوفة نوعها Shape، ونقدر نحط فيها أي كلاس فرعي منه.

8–10 نفس السطر `s.display()` يطلع نتيجة مختلفة حسب الكائن: الدائرة $\pi \times 2^2 \approx 12.57$ والمستطيل $4 \times 5 = 20.00$.

لو بكرة أضفنا Triangle، ما نحتاج نغيّر سطر واحد في الحلقة. **هذي قوة التجريد.**

8 الملخص
Summary

مفهوم Abstraction
نُظهر "ماذا" ونخفي "كيف". نركّز على الأساسيات ونخبّي التفاصيل المعقدة.

الكلمة abstract
تعني "ناقص". تنكتب قبل class أو قبل الدالة.

Abstract Class
كلاس عام يكون أساس لغيره. فيه متغيرات و Constructor ودوال مجردة ومكتملة.

Abstract Method
توقيع فقط بدون جسم، ينتهي بـ `;` . الكلاس الفرعي مُلزَم يكملها.

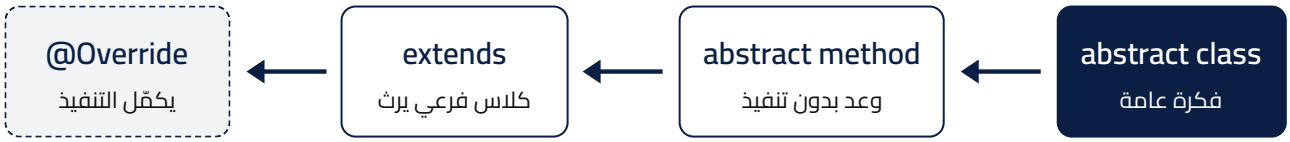
Concrete Method
دالة عادية لها جسم، تُورث وتُستخدم جاهزة.

الوراثة
`class Circle extends Shape` ثم إعادة كتابة الدوال المجردة مع `@Override` .

قيود إنشاء Objects
`new Shape()` ✗ خطأ، لكن
`Shape s = new Circle()` ✓ صحيح.

تطبيق التجريد
نتعامل مع كائنات مختلفة بنفس الكود، ونضيف أنواع جديدة بسهولة.

الخريطة الذهنية في سطر واحد



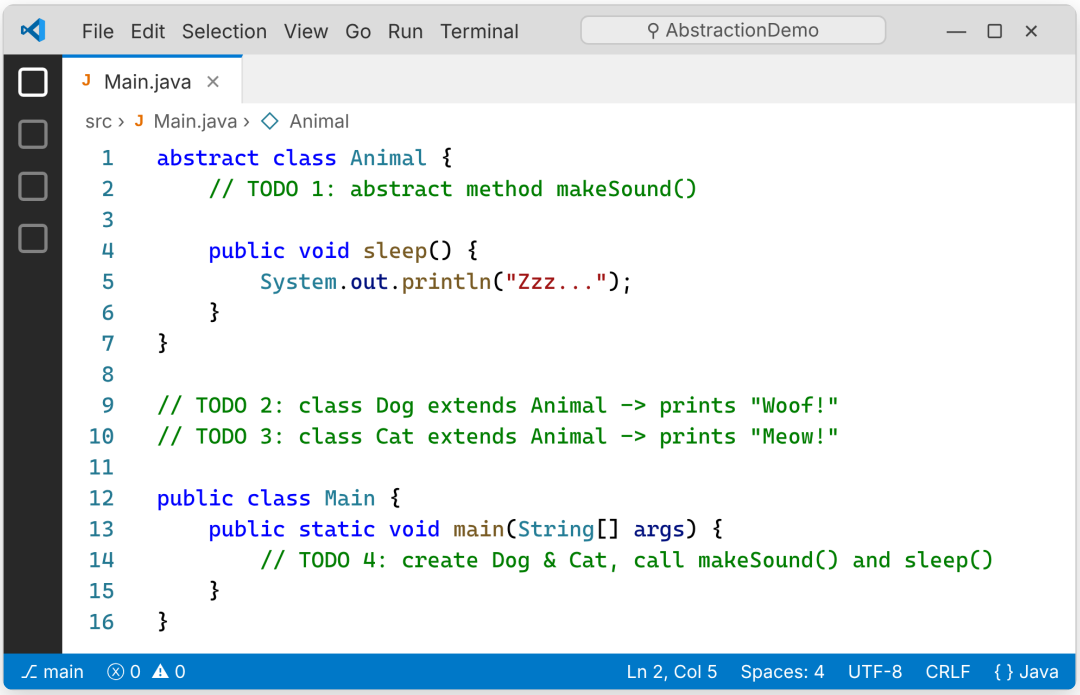
جملة وحدة تحفظها
"الـ Abstract Class يقول **إيش** لازم يصير، والكلاس الفرعي يقول **كيف** يصير"

9 تمرين تطبيقي

نحلّه سوا · Exercise

المطلوب: أكمل الكود التالي بحيث:

- يحتوي الكلاس المجرد Animal على دالة مجردة `makeSound()` لا ترجع قيمة.
- ينشئ كلاسين Dog و Cat يرثان من Animal، كل واحد يطبع صوته.
- في `main()` أنشئ كلبًا وقطة، وناد `makeSound()` لكل واحد، ثم `sleep()` للكلب.



الكود المبدئي للتمرين

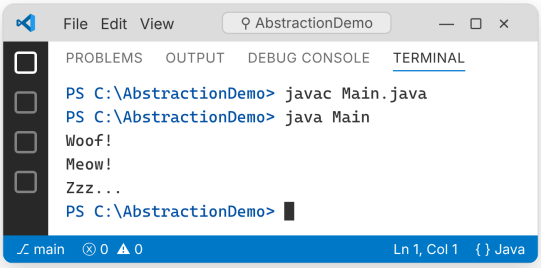
تلميح

الدالة المجردة في Animal:

```
public abstract void makeSound();
```

وفي Dog و Cat:

```
@Override
public void makeSound()
```



الناتج المتوقع

مساحة الحل

اكتب الحل هنا خطوة بخطوة مع الطلاب: **1** الدالة المجردة ← **2** الكلاس Dog ← **3** الكلاس Cat ← **4** الدالة main().